

NXP S32K3X8-EVB

Qemu support extension

Embedded Operating Systems - a.y. 2024/2025

Group 2

QEMU simulation:

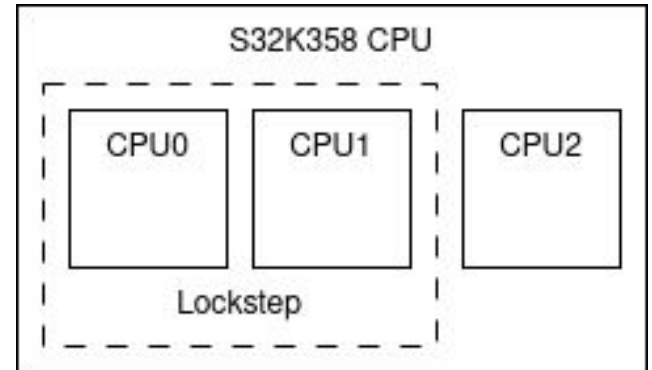
Machine & peripherals

Machine & CPU

Uses ARMv7M device provided by QEMU

2 CPUs are implemented, but can be modified for 4 CPUs

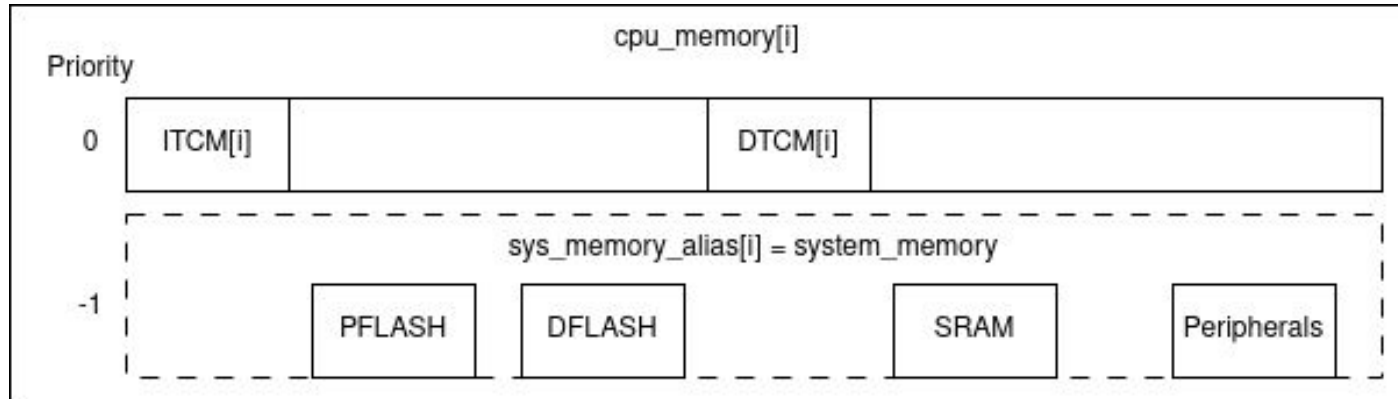
The number of CPUs are set by `-smp` (default 1)



Memory regions

Each core has a specific view of memory, with private and shared regions

Aliases are used for the shared regions



Note: diagram not to scale

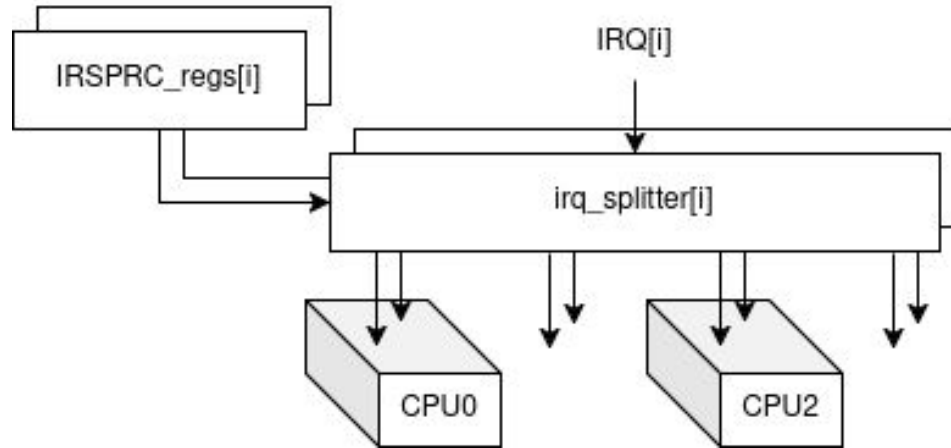
Memory regions

Memory	Base address	Size	Private/shared	Notes
SRAM	0x20400000	768 KB	Shared	SRAM 0,1,2 modeled together
Program flash	0x00400000	8 MB	Shared	4 blocks modeled together
Data flash	0x10000000	128 KB	Shared	
ITCM	0x00000000	64 KB	Private	
DTCM	0x20000000	128 KB	Private	

Interrupt routing

In the real board: IRSPRC register decide which cores each interrupt is routed to

In emulation: we use splitters



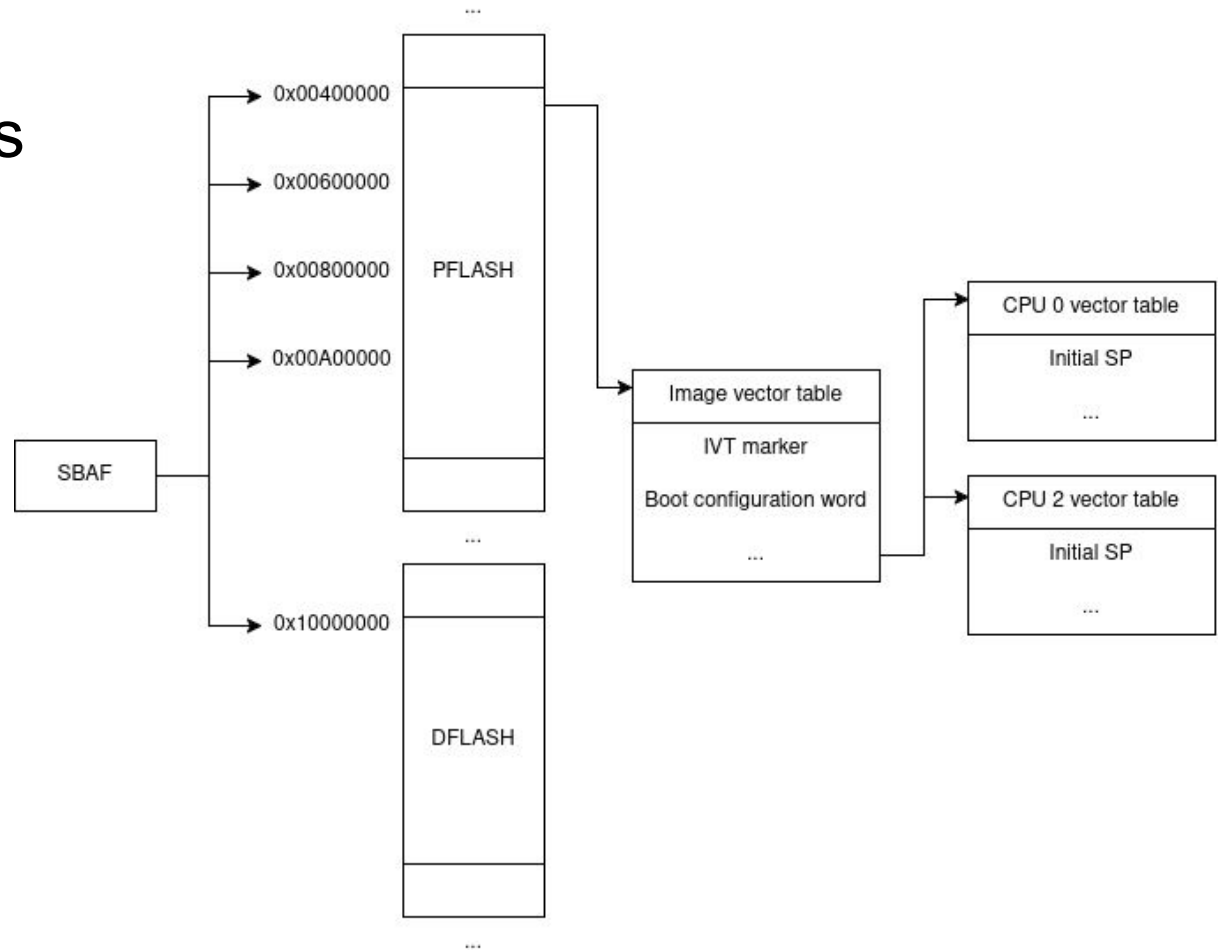
SBAF boot process

In the real board: the SBAF looks for IVT at Flash addresses:

```
0x00400000 (PFLASH block 0)
0x00600000 (PFLASH block 1)
0x00800000 (PFLASH block 2)
0x00A00000 (PFLASH block 3)
0x10000000 (DFLASH)
```

In emulation: we emulate the SBAF by parsing ELF file first

SBAF boot process



Timer Peripheral Implementation

Timer emulated: Periodic Interrupt Timer (PIT)

Functionality:

- Counts down from a start value and reloads automatically.
- Generates interrupts at timeout; each timer is independent.

4 Timers mapped to specific memory regions with separate IRQs (96–99)

PIT 0	0x400B0000	0x400B3FFF
PIT 1	0x400B4000	0x400B7FFF
PIT 2	0x402FC000	0x402FFFFFF

Timer Peripheral Implementation

Registers:

- VALUE: writes directly set the current timer count.
- RELOAD: writes update the current count and the reload value.
- INTSTATUS: writes clear interrupt flags using write-1-to-clear semantics.
- CTRL: register writings start or stop the timer.

Defined key functions:

- read() → register reads
- write() → config + control
- tick() → triggers interrupt

Board definition:

- Child object
- Connection to the system clock
- sysbus_realize()
- Mapping
- Interrupt splitter

UART

Device Registration:

- Instantiating 16 UART as a child of the MCU with `object_initialize_child()` function.
- Connecting the frontend to backend with `qdev_prop_set_chr()` function.
- `sysbus_realize()`.
- `sysbus_mmio_map()`
- `sysbus_connect_irq()`.

UART

Implementation:

- Guest code → Write into UART memory → Handler → Switch Case.
- For the TX Path, `s32k3x8_uart_write()` → `qemu_chr_fe_write_all()`.
- For the RX Path, `uart_can_receive()` → `uart_rx()` → Timer Callback.
- The RX path has a baud-rate delay implemented.
- Testing; Interrupts, RX, TX, Baud-Rate.

FreeRTOS & program

FreeRTOS - configuration (makefile, linker, startup)

- Correct **FreeRTOS port layer** selected: `portable/GCC/ARM_CM7/r0p1`
- FPU support enabled via `-mfpv5-sp-d16 -mfloat-abi=hard`
- `MACHINE := S32K3X8 CPU := cortex-m7`
- Memory regions mapped according to description given in reference manual
 - PFLASH
 - DFLASH
 - RAM
 - ITCM
 - DTCM
- Interrupt vector table for both CPUs
 - maps all core exceptions and peripheral interrupts—including those used by FreeRTOS like PendSV, SysTick, and SVC

linker_rtos.ld (2 cores) and mps2_m7.ld (single core)

```
MEMORY
{
    PFLASH (xr) : ORIGIN = 0x00400000, LENGTH = 8M
    DFLASH (xr) : ORIGIN = 0x10000000, LENGTH = 128K
    RAM (rw) : ORIGIN = 0x20400000, LENGTH = 768K
    ITCM (rw) : ORIGIN = 0x00000000, LENGTH = 64K
    DTCM (rw) : ORIGIN = 0x20000000, LENGTH = 128K
}

_Min_Heap_Size = 0x8 ; /* Not used as building heap_4.c */
_Min_Stack_Size = 0x400 ; /* Required amount of stack. Used by main(), then r
_estack = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
    .boot_ivt :
    {
        /* __vector_table = .; */
        . = ALIGN(128); /* This is very important, see below */
        KEEP(*(.boot_ivt))
    } > PFLASH

    /* The individual vector table must be 7-bit aligned in memory.
     * Therefore, we need to add at least ALIGN(128) between every
     * CPU's vector table. See also:
     * https://developer.arm.com/documentation/dui0646/c/Cortex-M7-Peripherals/System-Memory-Block-Descriptions
     */
    .isr_vector_0 :
    {
        . = ALIGN(128);
        KEEP(*(.isr_vector_0))
    } > PFLASH

    .isr_vector_1 :
    {
        . = ALIGN(128);
        KEEP(*(.isr_vector_1))
    } > PFLASH
```

```
MEMORY
{
    FLASH (xr) : ORIGIN = 0x00000000, LENGTH = 64K /* to 0x00003FFF = 0x007FFFFF */
    RAM (rw) : ORIGIN = 0x20400000, LENGTH = 768K /* to 0x21FFFFFF = 0xFFFFF */
}
ENTRY(Reset_Handler)

_Min_Heap_Size = 0x8 ; /* Not used as building heap_4.c */
_Min_Stack_Size = 0x400 ; /* Required amount of stack. Used by main(), then r
_estack = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
    .isr_vector :
    {
        __vector_table = .;
        KEEP(*(.isr_vector))
        . = ALIGN(4);
    } > FLASH
```

FreeRTOS Config

Interrupt Priority Configuration Problem (FreeRTOSConfig.h)

```
- #define configMAX_SYSCALL_INTERRUPT_PRIORITY      ( 4 )  
+ #define configPRIO_BITS 4  
+ #define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 0b0001  
+ #define configMAX_SYSCALL_INTERRUPT_PRIORITY      ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS) )
```

- System initially halted at `configASSERT( ucMaxSysCallPriority )`
- Cortex-M7 NVIC requires left-aligned priority encoding
- Used unshifted literal (4)
- `configASSERT( ucMaxSysCallPriority )` recognized priority as 0, thus halting everything

Simple FreeRTOS App

```
(gdb) target remote :1234
Remote debugging using :1234
Reset_Handler () at startup.c:83
warning: Source file is more recent than executable.
83      {
(gdb) break main
Breakpoint 1 at 0xd2c: file main.c, line 19.
(gdb) p gBlinkCounter
$2 = 0
(gdb) continue
Continuing.

Breakpoint 1, main (argc=0, argv=0x0 <isr_vector>) at main.c:19
19      xTaskCreate(
(gdb) p gBlinkCounter
$3 = 0
(gdb) continue
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x00000278 in prvCheckTasksWaitingTermination () at ../FreeRTOS/FreeRTOS/Source/tasks.c:6172
6172     }
(gdb) p gBlinkCounter
$4 = 6
```

FreeRTOS - program

- The sample FreeRTOS program was developed in such a way to demonstrate correct implementation of the assigned devices: UART and Timers
- The app basically does a slow computation of the Fibonacci series while keeping the CPU busy. Since it uses `uint32_t` to store values, each time an overflow would be reached, the computation restarts incrementing the iteration count.
- The execution of this simple program involves little features testing correct support of the S32K3x8 EVB mentioned devices

FreeRTOS - program

- Timers:
 - A software timer can be optionally created to automatically stop the series computation at the beginning of the program through keyboard input
 - Timer 0 interrupt: used to dump register context on screen
 - Timer 1 interrupt: used to reveal the last computed Fibonacci number and iteration counter
 - Timer 2 interrupt: used to dump memory content on screen
- UART:
 - On-screen output is produced constantly during the execution of the program
 - Keyboard input is used to let user decide if the task must be automatically deleted after a software timer countdown

License

[NXP S32K3X8-EVB Qemu support extension](#)

© 2025 by EOS - Group 2 is licensed under CC BY-NC 4.0.

To view a copy of this license, visit <https://creativecommons.org/licenses/by/4.0/>

